The Diagrams Explained in Simple Words

Team 032 "Himalayan Coders" | Student Mentoring and Career Support Platform | Milestone 2 review guide for the team (written for Adnan's review)

> **How to use this guide.** Read Part 1 first (the idea in plain words). Then read Part 2 (class diagram) and Part 3 (database diagram) with the picture open next to the text. Part 4 is a worked example that follows one student through the whole system. Part 5 lists review questions you can answer yes or no. You do not need to know UML or SQL beforehand; every new word is explained when it first appears.

Part 1. The idea behind both diagrams

What does our system do?

Our platform helps students of the programme get a mentor for career guidance. There are three kinds of people who use it:

Person	What they do on the platform
Student	Registers, fills a profile (career goals, skills, preferences, when they are free), accepts or declines the mentor proposed for them, and later gives feedback or raises a concern.
Mentor	Registers, fills a profile (qualifications, experience, areas of expertise), sets the time slots when they are free, and accepts or declines a match that was proposed to them.
Administrator (coordinator)	Checks mentor profiles, proposes which mentor suits which student, confirms the match once both sides have accepted (or cancels it), and follows the progress of every match.

The one rule that shapes everything: "client-mediated"

Students and mentors **cannot search or browse each other**. A student cannot pick a mentor and a mentor cannot pick a student. Only the administrator makes the pairing. The pairing is called a **Match**. This is why you will see a **Match** box in the middle of both diagrams: almost everything connects to it.

> **Why did we choose this?** It matches the problem statement (the platform acts as a middle party) and keeps control with the coordinator, who knows the students. It also means the screens are simple: a student never sees a list of mentors, only the one mentor assigned to them.

Why two diagrams?

Diagram	Question it answers	Who cares most
Class diagram	What are the "things" in our program, what information does each hold, and what can each one *do*?	Developers writing code
ER diagram (Entity Relationship)	How do we store the information in the database tables, and how are the tables linked?	Whoever builds the database and the APIs

They describe the same system from two angles, so the names are almost the same (Student, Mentor, Match, Feedback and so on). The class diagram adds *actions* (like "accept a match"). The ER diagram adds *database details* (like keys that link the tables).

How to read the lines

What you see	What it means
A box with a dark top	One "thing" (a class) or one database table. The top is its name.
Numbers on a line, like `1`, `N`, `0..*`	How many. `1` = exactly one. `N` or `*` = many. `0..*` = zero or many. Example: `1` on the Student side and `0..*` on the Match side means one student can be in zero, one or many matches.
Line with a hollow white triangle (class diagram)	"Is a kind of". Student, Mentor and Administrator are each a kind of User.
Dashed line with an arrow	"Uses". Match uses the list of allowed statuses.
`-` before a name	Private: information kept inside the object.
`+` before a name	Public: an action others can ask the object to do.
`PK` (ER diagram)	Primary key: a number that uniquely identifies each row, like a roll number.
`FK` (ER diagram)	Foreign key: a number that points to a row in another table, like writing a student's roll number on a form to say whose form it is.

Class Diagram: Student Mentoring Platform (client-mediated)

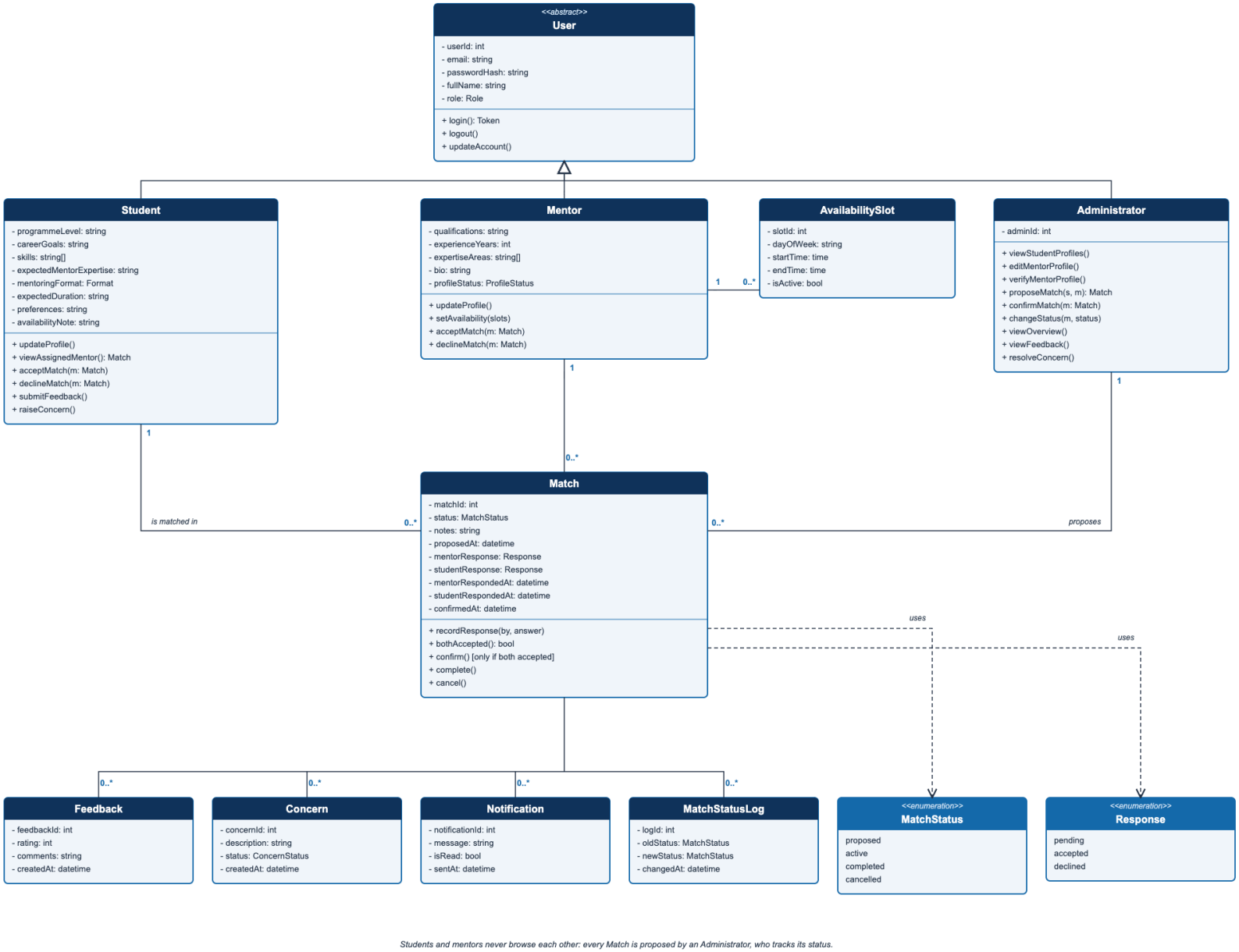


Figure 1. Class diagram. Keep this page open while reading Part 2.

Part 2. The class diagram, box by box

Look at the picture in three layers: the **top** (User and its three kinds), the **middle** (Match), and the **bottom** (things that belong to a Match).

Layer 1: User and its three kinds

User (marked `<<abstract>>`) is the common base. "Abstract" simply means we never create a plain User; every real person is a Student, a Mentor or an Administrator. User holds what every person has: an id, email, password (stored scrambled, as a hash), full name and a role. Its actions are `login()`, `logout()` and `updateAccount()`.

The three boxes under it, joined by hollow triangles, are kinds of User. They automatically get everything User has, and add their own:

Class	Extra information it keeps	Actions it can do
Student	programme level, career goals, skills, expected mentor expertise, preferred format (video, phone, messaging), expected duration, preferences, availability note	`updateProfile()`, `viewAssignedMentor()` (see the mentor chosen for them), `acceptMatch(m)`, `declineMatch(m)`, `submitFeedback()`, `raiseConcern()`
Mentor	qualifications, years of experience, expertise areas, short bio, profile status (pending or verified)	`updateProfile()`, `setAvailability(slots)`, `acceptMatch(m)`, `declineMatch(m)`
Administrator	an admin id	`viewStudentProfiles()`, `editMentorProfile()`, `verifyMentorProfile()`, `proposeMatch(s, m)`, `changeStatus(m, status)`, `viewOverview()`, `viewFeedback()`, `resolveConcern()`

AvailabilitySlot (to the right of Mentor) is one free time window: a day of the week, a start time and an end time. The line says one Mentor has zero or many slots (`1` to `0..*`). A new mentor starts with zero slots and adds them later.

> **Notice what is missing.** Student has no "search mentors" action and Mentor has no "pick student" action. Only the Administrator has `proposeMatch`, and only the Administrator can view all student profiles. Feedback is given by the student only, because the Milestone 1 stories have no mentor feedback. That is the client-mediated rule written into the diagram.

Layer 2: Match, the heart of the system

Match is one pairing of one student and one mentor. It stores a match id, the status, optional notes, when it was proposed, the **mentor's response** and the **student's response** (each with its date), and when the coordinator confirmed it. Its actions are `recordResponse(by, answer)`, `bothAccepted()`, `confirm()` (allowed only when both accepted), `complete()` and `cancel()`.

The lines into Match, all with `0..*` on the Match side:

- **Student to Match:** one student can appear in many matches over time (for example if the first one is cancelled and a new mentor is proposed).
- **Mentor to Match:** one mentor can mentor several students.
- **Administrator to Match:** one administrator can propose many matches. This is the line labelled "proposes".

Layer 3: things that hang off a Match

Class	What it is
Feedback	A rating (1 to 5) and comments that the student gives about a match. The administrator can read it. A match has zero or many feedback entries.
MatchStatusLog	One entry for every status change: the old status, the new status and when it happened. It is the history of a match.
Concern	A problem someone raises about a match (for example "my mentor has not replied"). It has a status (open or resolved) so the administrator can follow it up.
Notification	A message sent to a user (for example "You have a new match proposal"). It stores when it was sent and whether it was read. When a match becomes active, the student's message also tells the next step.
MatchStatus (`<<enumeration>>`)	An "enumeration" is a fixed list of allowed values. Here: proposed, active, completed, cancelled. A match can only ever hold one of these four words. It is drawn with a dashed arrow because Match *uses* it.
Response (`<<enumeration>>`)	The answer of the mentor or of the student to a proposed match: pending (not answered yet), accepted or declined. Match keeps one Response for the mentor and one for the student.

The life of a Match (status flow)

This is the most important rule for the developers and the testers. A match moves forward in these steps only. The mentor and the student first answer the proposal; the coordinator then **confirms** it, and confirming is what makes the match **active**. There is no separate "confirmed" status.

proposed → **active** → **completed**

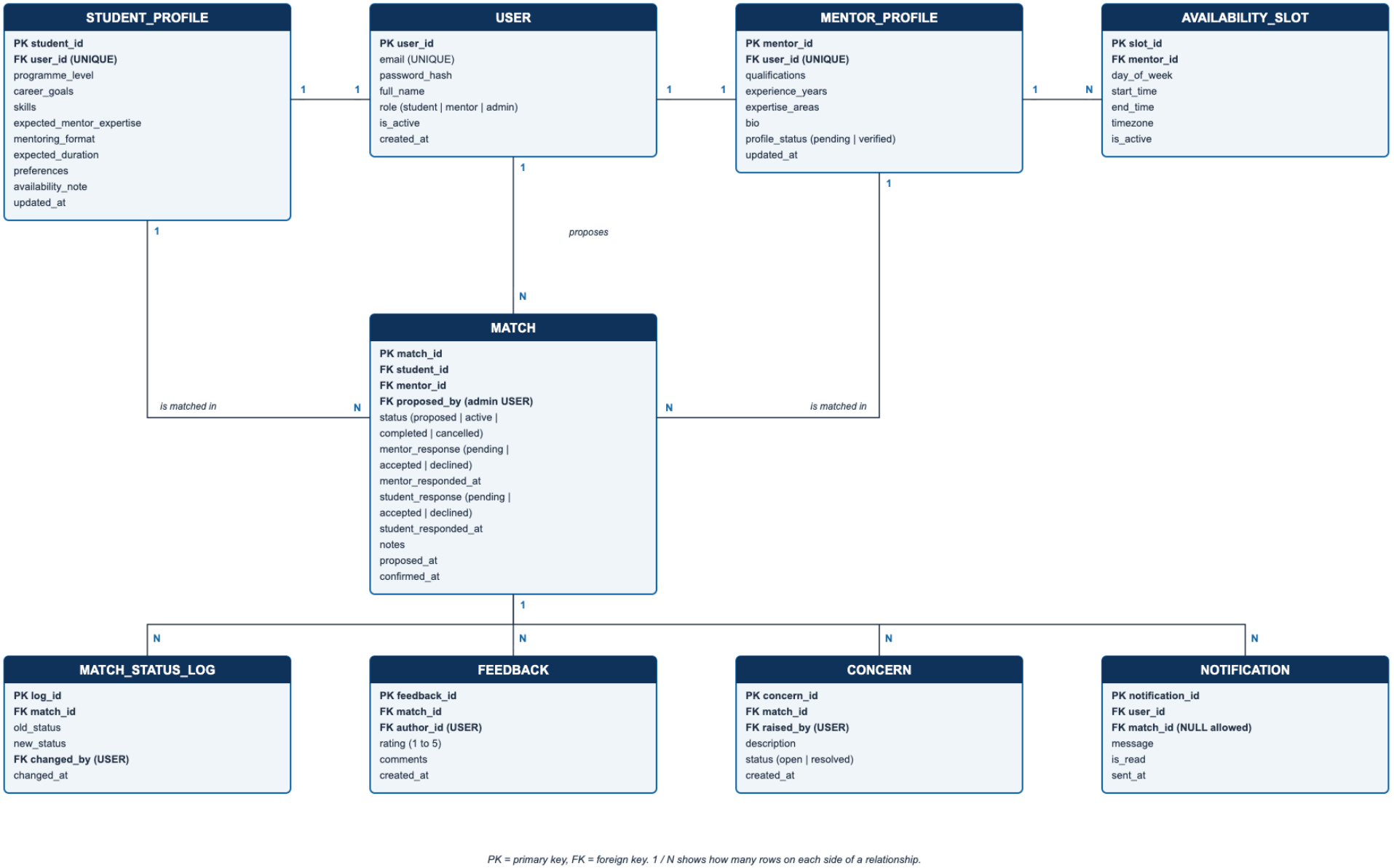
Any step before **completed** can also go to → **cancelled**

From	Can go to	Meaning in real life
proposed	active or cancelled	The admin suggested the pair and **both the mentor and the student are notified**. Each accepts or declines. Only when **both have accepted** can the admin confirm the match (it becomes active). If either side declines, or the admin simply does not want the match, the admin cancels it.
active	completed or cancelled	Mentoring is running. It ends normally (completed) or stops early (cancelled).
completed, cancelled	nothing	These are final. A finished or cancelled match is never reopened; a new match is proposed instead.

> **Who answers what.** The two answers (Response) are separate from the status. While the status is *proposed*, the mentor's answer and the student's answer can each be pending, accepted or declined. The status changes to *active* only by the admin's confirmation, and the system refuses to confirm unless both answers are accepted.

The prototype frontend already follows exactly this table, so the testers can check it by trying to make an illegal jump (for example proposed straight to completed) and confirming it is not allowed.

Entity Relationship Diagram: Student Mentoring Platform (client-mediated)



PK = primary key, FK = foreign key. 1 / N shows how many rows on each side of a relationship.

Figure 2. Entity Relationship diagram. Keep this page open while reading Part 3.

Part 3. The ER (database) diagram, table by table

Every box here is a **table** in the database. Think of each table as an Excel sheet: the field names are the column headings and each person or record is one row. Tables are linked by keys so we do not have to copy the same data twice.

Why is there a separate USER table and a STUDENT_PROFILE table?

Login details (email, password, role) are the same for everybody, so they live once in **USER**. The details that only students have live in **STUDENT_PROFILE**, and the details that only mentors have live in **MENTOR_PROFILE**. Each profile has a `user_id` that points back to its USER row. The note "UNIQUE" means one user can have only one profile. This is the database version of "a Student is a kind of User" from the class diagram.

The top row: people and their profiles

Table	Columns (plain meaning)	Notes
USER	user_id (PK), email (unique), password_hash, full_name, role (student, mentor or admin), is_active, created_at	One row per person who can log in. The password is stored as a scrambled hash, never as plain text. `role` decides which screens the person sees.
STUDENT_PROFILE	student_id (PK), user_id (FK, unique), programme_level, career_goals, skills, expected_mentor_expertise, mentoring_format, expected_duration, preferences, availability_note, updated_at	Linked 1-to-1 with USER. The student's answer to a proposal is stored on the MATCH, not here.
MENTOR_PROFILE	mentor_id (PK), user_id (FK, unique), qualifications, experience_years, expertise_areas, bio, profile_status, updated_at	Linked 1-to-1 with USER. `profile_status` is pending until the admin verifies it.
AVAILABILITY_SLOT	slot_id (PK), mentor_id (FK), day_of_week, start_time, end_time, timezone, is_active	One row per free time window. One mentor has many slots (the 1 to N line).

The middle: MATCH

Table	Columns	Notes

MATCH	match_id (PK), student_id (FK), mentor_id (FK), proposed_by (FK to an admin USER), status, mentor_response, mentor_responded_at, student_response, student_responded_at, notes, proposed_at, confirmed_at	One row per pairing. It has **three** foreign keys: which student, which mentor and which admin proposed it. `status` holds one of the four allowed words. The two responses are pending, accepted or declined, each with the time it was given. `confirmed_at` is filled when the admin confirms.

The three lines into MATCH in the picture are therefore: student to match (1 to N), mentor to match (1 to N) and admin user to match (1 to N, labelled "proposes").

The bottom row: records about a match

Each of these has a `match_id` foreign key, so one match can have many of them (1 to N).

Table	Columns	Why we need it
MATCH_STATUS_LOG	log_id (PK), match_id (FK), old_status, new_status, changed_by (FK to USER), changed_at	A history. Every time a status changes, one row is added. This lets the admin see who changed what and when, and it is useful evidence during testing.
FEEDBACK	feedback_id (PK), match_id (FK), author_id (FK to USER), rating 1 to 5, comments, created_at	The student's feedback on the match, which the administrator can read. `author_id` says who wrote it.
CONCERN	concern_id (PK), match_id (FK), raised_by (FK to USER), description, status (open or resolved), created_at	Problems reported about a match, followed up by the admin.
NOTIFICATION	notification_id (PK), user_id (FK), match_id (FK, may be empty), message, is_read, sent_at	Messages shown to a user. `match_id` may be empty because some notices (for example "your mentor profile was verified") are not about any match.

Class diagram and ER diagram side by side.

Class diagram	ER diagram table	Difference to remember
User, Student, Mentor, Administrator	USER, STUDENT_PROFILE, MENTOR_PROFILE	Administrator has no table of its own: an admin is just a USER whose role is "admin".
AvailabilitySlot	AVAILABILITY_SLOT	Same.
Match and its enumeration	MATCH (status column)	The enumeration becomes the allowed values of the status column.
Feedback, Concern, Notification	FEEDBACK, CONCERN, NOTIFICATION	Same.
MatchStatusLog	MATCH_STATUS_LOG	Same: the history of every status change.

Part 4. Worked example: one student from start to finish

Follow this story and point at the tables and classes as you go. The names are made up.

1. **Riya registers as a student.** A row is added to USER (role = student) and one to STUDENT_PROFILE with her goals and skills. *(Classes: User, Student.)*
2. **Mr. Rao registers as a mentor.** A row goes to USER (role = mentor) and MENTOR_PROFILE with profile_status = pending. He adds two free slots, which become two rows in AVAILABILITY_SLOT. *(Classes: Mentor, AvailabilitySlot.)*
3. **The admin verifies Mr. Rao.** profile_status changes from pending to verified. *(Action: verifyMentorProfile().)*
4. **The admin proposes a match.** The admin reads both profiles, then creates a row in MATCH with Riya's student_id, Mr. Rao's mentor_id, proposed_by = the admin, status = **proposed**, and both responses = **pending**. A row is added to MATCH_STATUS_LOG and a NOTIFICATION row is created for **both** Mr. Rao and Riya. *(Action: proposeMatch(s, m).)*
5. **Mr. Rao accepts.** mentor_response becomes **accepted** and mentor_responded_at is filled. The admin is notified. The match is still **proposed**, so Riya cannot see an active mentor yet. *(Action: acceptMatch(m) on Mentor.)*
6. **Riya accepts.** student_response becomes **accepted**. Now both have accepted, so the admin is notified that the match is ready to confirm. *(Action: acceptMatch(m) on Student.)* If either of them had declined, the admin would cancel the match and propose someone else.
7. **The admin confirms.** status becomes **active**, confirmed_at is filled, another log row is added, and Riya and Mr. Rao each get a notification with the next step. *(Action: confirmMatch(m).)* The system would refuse this step if either answer were still pending or declined.
8. **Mentoring begins.** The match is **active**. Riya opens her mentor page and sees Mr. Rao. *(Action: viewAssignedMentor().)*
9. **During mentoring.** Riya can raise a concern (a row in CONCERN, status open). The admin resolves it (status resolved). *(Actions: raiseConcern(), resolveConcern().)*
10. **Mentoring ends.** status becomes **completed**. Riya gives feedback (a row in FEEDBACK with a rating from 1 to 5) and the admin can read it. The match is now closed for ever.

Part 5. Review checklist for Adnan

Tick each question yes or no. Write any "no" in the meeting notes with a short reason, so the team can fix it before submission.

#	Question	Yes / No
1	Can a student or mentor browse the other side anywhere in the diagrams? (Answer should be **no**.)	

2	Is the Administrator the only one who can propose a match?	
3	Does the flow cover: propose (both notified), mentor and student each accept or decline, admin confirms only when both accepted (or cancels), complete, cancel?	
4	Can every field in the class diagram be found in an ER table (or is its absence explained, like Administrator)?	
5	Does every user story in the Milestone 1 report have a class action and a place to store its data? Check these in particular: US-S04 and US-S05 (expected expertise, format, duration), US-S08 (the student's response with its date), US-C01 and US-C05 (admin views students and edits mentors), US-S10 (feedback readable by the admin).	
6	Can a student have more than one match over time (for example after a cancellation)?	
7	Is it clear who changed a status and when (MATCH_STATUS_LOG)?	
8	Are the multiplicities (1, N, 0..*) believable? Example: one mentor has many slots; one match has one student and one mentor.	
9	Is anything missing that the users would expect (for example the mentor's reason for declining, a calendar, chat)? Note it; do not add it without the team's approval.	

Questions the team still needs to decide

- If the mentor or the student declines, should the system cancel the match by itself, or leave the decision to the admin? (The prototype leaves it to the admin, who can cancel and propose a new match.)
- What may each side see before accepting? (The prototype shows the student a short summary of the proposed mentor, and shows the mentor a short summary of the student.)
- Which database will store these tables? This is part of the tech stack decision in Jira card KAN-17.

Glossary

Word	Meaning
Class	A blueprint for a "thing" in the program, with its information and actions.
Entity / table	A kind of record kept in the database, like a sheet in Excel.
Primary key (PK)	The unique number of a row.
Foreign key (FK)	A number in one table that points to a row in another table.
Multiplicity	How many items sit on each side of a line (1, N, 0..*).
Inheritance	"Is a kind of". The child gets everything the parent has.

Abstract class	A class that is never created on its own, only through its kinds.
Enumeration	A fixed list of allowed values.
Hash	A scrambled form of a password that cannot be turned back into the password.
Client-mediated	Pairing is done by a middle party (the admin), not chosen by the two sides themselves.